

Program

```
<!DOCTYPE html>
<html>
<head>
<title>Credit System Ultra</title>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<style>
body {
  margin:0;
  font-family:Arial;
  background:linear-gradient(135deg,#0f172a,#1e293b);
  color:white;
}
.card {
  backdrop-filter: blur(10px);
  background: rgba(255,255,255,0.1);
  border-radius: 15px;
  padding:20px;
  margin:10px;
}
.container {display:flex; flex-wrap:wrap;}
.sidebar {
  width:200px;
  background:#111827;
  min-height:100vh;
  padding:15px;
}
.sidebar button {
  width:100%;
  margin:8px 0;
  padding:10px;
  background:#6366f1;
  border:none;
  color:white;
}
.main {flex:1; padding:20px;}
```

```

.main {flex:1; padding:20px;}
input,select {
  width:90%;
  padding:10px;
  margin:5px;
}
button {
  padding:10px;
  margin:5px;
  background:#6366f1;
  border:none;
  color:white;
}
.green {color:#22c55e;}
.red {color:#ef4444;}
</style>
</head>

<body>
<div id="app"></div>

<script>

let users = JSON.parse(localStorage.getItem("users") || "[]");
let currentUser = JSON.parse(localStorage.getItem("currentUser") || "null");

// 🗝️ FAKE TOKEN
function generateToken(user){
  return btoa(user.email + ":" + Date.now());
}

// ===== AUTH =====

function signup(){
  let email = document.getElementById("email").value;
  let password = document.getElementById("password").value;

  let user = {

```

```
function signup(){
  let email = document.getElementById("email").value;
  let password = document.getElementById("password").value;

  let user = {
    email,password,
    limit:50000,
    balance:50000,
    budget:10000,
    transactions:[]
  };

  users.push(user);
  localStorage.setItem("users", JSON.stringify(users));

  alert("Registered ");
  renderLogin();
}

function login(){
  let email = document.getElementById("email").value;
  let password = document.getElementById("password").value;

  currentUser = users.find(u => u.email === email && u.password === password);

  if(currentUser){
    localStorage.setItem("currentUser", JSON.stringify(currentUser));
    renderDashboard();
  } else {
    alert("Invalid Login ");
  }
}
```

```

function logout(){
  localStorage.removeItem("currentUser");
  currentUser = null;
  renderLogin();
}

// ===== DASHBOARD =====

function renderDashboard()
{
  let income=0, expense=0;

  currentUser.transactions.forEach(t=>{
    if(t.type==="income") income+=t.amount;
    else expense+=t.amount;
  });

  document.getElementById("app").innerHTML=`
<div class="container">

<div class="sidebar">
  <button onclick="renderDashboard()">Dashboard</button>
  <button onclick="renderTransaction()">Transaction</button>
  <button onclick="renderBudget()">Budget</button>
  <button onclick="renderChart()">Analytics</button>
  <button onclick="logout()">Logout</button>
</div>

<div class="main">

  <div class="card">
    <h3>📇 Card</h3>
    <p>Limit: ₹${currentUser.limit}</p>
    <p>Balance: ₹${currentUser.balance}</p>
  </div>

```



```
<div class="card">
  <h3>🏠 Card</h3>
  <p>Limit: ₹${currentUser.limit}</p>
  <p>Balance: ₹${currentUser.balance}</p>
</div>
```

```
<div class="card">
  <h3>📊 Stats</h3>
  <p class="green">Income: ₹${income}</p>
  <p class="red">Expense: ₹${expense}</p>
  <p>Budget: ₹${currentUser.budget}</p>
</div>
```

```
</div>
```

```
</div>`;
```

```
}
```

```
// ===== TRANSACTION =====
```

```
function renderTransaction(){
  document.getElementById("app").innerHTML=`
  <div class="card">
    <h2>Transaction</h2>
    <input id="amount" placeholder="Amount">
    <select id="type">
      <option value="expense">Expense</option>
      <option value="income">Income</option>
    </select>
    <button onclick="makeTransaction()">Submit</button>
    <button onclick="renderDashboard()">Back</button>
  </div>`;
}
```

```

function makeTransaction(){
  let amt = Number(document.getElementById("amount").value);
  let type = document.getElementById("type").value;

  let spent = currentUser.transactions
    .filter(t=>t.type==="expense")
    .reduce((a,b)=>a+b.amount,0);

  // 🔥 CORE LOGIC
  if(type==="expense" && spent + amt > currentUser.budget){
    let proceed = confirm("⚠ Budget exceeded! Proceed?");
    if(!proceed){
      alert("Cancelled ❌");
      return;
    }
  }

  if(type==="expense") currentUser.balance -= amt;
  else currentUser.balance += amt;

  currentUser.transactions.push({amount:amt,type});

  updateUser();
  alert("Transaction Success ✅");
  renderDashboard();
}

// ===== BUDGET =====

function renderBudget(){
  document.getElementById("app").innerHTML=`
<div class="card">
  <h2>Set Budget</h2>
  <input id="budget" placeholder="Enter Budget">
  <button onclick="saveBudget()">Save</button>
  <button onclick="renderDashboard()">Back</button>
</div>`;
}

```

```
function saveBudget(){
  currentUser.budget = Number(document.getElementById("budget").value);
  updateUser();
  alert("Saved ✅");
  renderDashboard();
}
```

```
// ===== CHART =====
```

```
function renderChart(){
  document.getElementById("app").innerHTML=`
  <div class="card">
    <h2>Analytics</h2>
    <canvas id="chart"></canvas>
    <button onclick="renderDashboard()">Back</button>
  </div>`;

  let data = currentUser.transactions.map(t=>t.amount);

  new Chart(document.getElementById("chart"),{
    type:"pie",
    data:{
      labels:["Transactions"],
      datasets:[{data}]
    }
  });
}
```

```
// ===== STORAGE =====
```

```
function updateUser(){
  users = users.map(u =>
    u.email === currentUser.email ? currentUser : u
  );

  localStorage.setItem("users", JSON.stringify(users));
  localStorage.setItem("currentUser", JSON.stringify(currentUser));
}
```



```
// ===== LOGIN UI =====

function renderLogin(){
  document.getElementById("app").innerHTML=`
    <div class="card" style="width:300px;margin:100px auto">
      <h2>Login</h2>
      <input id="email" placeholder="Email">
      <input id="password" type="password" placeholder="Password">
      <button onclick="login()">Login</button>
      <button onclick="renderSignup()">Signup</button>
    </div>`;
}

function renderSignup(){
  document.getElementById("app").innerHTML=`
    <div class="card" style="width:300px;margin:100px auto">
      <h2>Signup</h2>
      <input id="email">
      <input id="password">
      <button onclick="signup()">Register</button>
      <button onclick="renderLogin()">Back</button>
    </div>`;
}

// ===== INIT =====

if(currentUser){
  renderDashboard();
}else{
  renderLogin();
}

</script>

</body>
</html>
```



```
Program;
<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: GET");
session_start();

include_once 'config/db.php';

if (!isset($_SESSION['user_id'])) {
    echo json_encode(array("message" => "Unauthorized."));
    exit;
}

$user_id = $_SESSION['user_id'];

$card_query = "SELECT card_number, credit_limit, balance FROM Cards WHERE user_id = '$user_id'";
$card_result = $conn->query($card_query);

$budget_query = "SELECT monthly_limit, current_spent FROM Budget WHERE user_id = '$user_id'";
$budget_result = $conn->query($budget_query);

if ($card_result->num_rows > 0 && $budget_result->num_rows > 0) {
    $card_data = $card_result->fetch_assoc();
    $budget_data = $budget_result->fetch_assoc();
    echo json_encode(array(
        "card_number" => $card_data['card_number'],
        "credit_limit" => $card_data['credit_limit'],
        "balance" => $card_data['balance'],
        "monthly_limit" => $budget_data['monthly_limit'],
        "current_spent" => $budget_data['current_spent']
    ));
} else {
    echo json_encode(array("message" => "Card or budget not found."));
}
?>
```

JavaScript:

```
import React, { useState } from 'react';
import api from '../api';

const Transaction = () => {
  const [merchant, setMerchant] = useState('');
  const [amount, setAmount] = useState('');
  const [category, setCategory] = useState('');
  const [alertMessage, setAlertMessage] = useState(null);
  const [showPrompt, setShowPrompt] = useState(false);

  const handleTransaction = async () => {
    try {
      const response = await api.post('/addTransaction.php', { merchant, amount, category });
      if (response.data.status === 'limit_exceeded') {
        setShowPrompt(true);
        setAlertMessage(response.data.message);
      } else {
        setAlertMessage(response.data.message);
        setShowPrompt(false);
      }
    } catch (error) {
      console.error('Error adding transaction:', error);
      setAlertMessage('Transaction failed.');
```

```
    }
  };

  const handleConfirmation = async (decision) => {
    // Simulate sending user decision to backend
    console.log(`User decided to: ${decision}`);
    setAlertMessage(`Transaction ${decision === 'proceed' ? 'approved' : 'cancelled'}.`);
    setShowPrompt(false);
  }

  return (
    <div className="container mt-4">
      <h2>Simulate Transaction</h2>
      <div className="form-group">
        <label>Merchant</label>
```

```

<div className="container mt-4">
  <h2>Simulate Transaction</h2>
  <div className="form-group">
    <label>Merchant</label>
    <input type="text" className="form-control" value={merchant} onChange={(e) => setMerchant(e.target.value)} />
  </div>
  <div className="form-group">
    <label>Amount (Rs.)</label>
    <input type="number" className="form-control" value={amount} onChange={(e) => setAmount(e.target.value)} />
  </div>
  <div className="form-group">
    <label>Category</label>
    <input type="text" className="form-control" value={category} onChange={(e) => setCategory(e.target.value)} />
  </div>
  <button className="btn btn-primary mt-3" onClick={handleTransaction}>Swipe Card</button>

  {alertMessage && <div className="alert alert-info mt-3">{alertMessage}</div>}

  {showPrompt && (
    <div className="modal show d-block" style={{ backgroundColor: 'rgba(0,0,0,0.5)' }}>
      <div className="modal-dialog">
        <div className="modal-content">
          <div className="modal-header">
            <h5 className="modal-title">Budget Limit Exceeded!</h5>
          </div>
          <div className="modal-body">
            <p>{alertMessage}</p>
          </div>
          <div className="modal-footer">
            <button type="button" className="btn btn-danger" onClick={() => handleConfirmation('decline')}>Decline</button>
            <button type="button" className="btn btn-success" onClick={() => handleConfirmation('proceed')}>Proceed</button>
          </div>
        </div>
      </div>
    </div>
  )}
</div>
);
};

```


PROGRAM:

```
CREATE DATABASE IF NOT EXISTS credit_card_system;  
USE credit_card_system;
```

1. Users Table

```
CREATE TABLE Users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIME  
);
```

2. Cards Table (Links each user to a virtual card)

```
CREATE TABLE Cards (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    card_number VARCHAR(16) UNIQUE NOT NULL,  
    credit_limit DECIMAL(10, 2) NOT NULL,  
    balance DECIMAL(10, 2) DEFAULT 0.00,  
    FOREIGN KEY (user_id) REFERENCES Users(id)  
);
```

3. Budget Table (Stores monthly budget limit for each user)

```
CREATE TABLE Budget (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    monthly_limit DECIMAL(10, 2) NOT NULL,  
    current_spent DECIMAL(10, 2) DEFAULT 0.00,  
    FOREIGN KEY (user_id) REFERENCES Users(id)  
);
```

3. Budget Table (Stores monthly budget limit for each user)

```
CREATE TABLE Budget (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    monthly_limit DECIMAL(10, 2) NOT NULL,  
    current_spent DECIMAL(10, 2) DEFAULT 0.00,  
    FOREIGN KEY (user_id) REFERENCES Users(id)  
);
```

4. Transactions Table (Records all transactions)

```
CREATE TABLE Transactions (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    merchant VARCHAR(255) NOT NULL,  
    amount DECIMAL(10, 2) NOT NULL,  
    date TIMESTAMP DEFAULT CURRENT_TIME,  
    status ENUM('Approved', 'Declined', 'Pending Confirmation') NOT NULL,  
    category VARCHAR(255) DEFAULT 'General',  
    FOREIGN KEY (user_id) REFERENCES Users(id)  
);
```